

Decision trees I: entropy and information gain

Ask the question that most reduces disagreement, split, and repeat: ID3 on the 14-day tennis table.

THE QUESTION

Which attribute should a decision tree test first, and how do we score that choice?

THE INTUITION

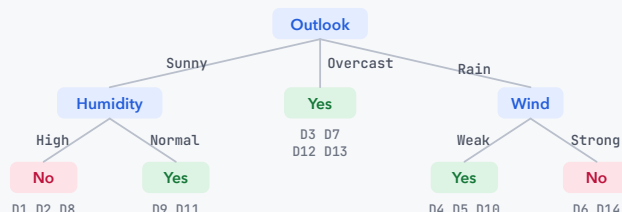
Examples that agree are easy to label. Entropy counts the disagreement in bits. Split on the attribute whose branches are purest, then repeat inside each branch.

KEY TAKEAWAY

Gain = parent entropy minus the size-weighted child entropy. On the tennis data Outlook wins, 0.247 bits.

14 days: 9 Yes, 5 No → $H(S) = 0.940$ → gain per attribute → split on Outlook → recurse on Sunny, Rain → pure leaves

1 A decision tree is a set of rules you can read



Interpretability builds trust: a deep network gives no reasons; a tree does. Each root-to-leaf path is a rule: if Outlook = Sunny and Humidity = High, do not play. Trees are used in medical diagnosis; Breiman's CART (1984) is the classic reference. Grey: the days that reach each leaf.

2 The tennis table: 14 days, 4 attributes

Day	Outlook	Temp	Humidity	Wind	Play
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

target: Play

 9 Yes 5 No
 $p(\text{Yes}) = 9/14$
 $p(\text{No}) = 5/14$
 $H(S) = 0.940$ bits
 all 14 alike would give $H = 0$

All inputs and the output, Play, are discrete. With 14 Yes (or 14 No) the problem is trivial; 9 against 5 is not. Less disagreement means an easier problem, so we need a number for disagreement.

3 The best tree is NP-complete to find

Check one tree

run 14 rows down it and count mistakes: fast

Find the best tree

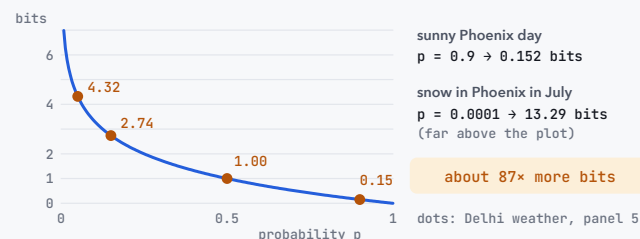
4.6×10^{17} labelled trees on just these 4 attributes

score every attribute → keep the best → split → repeat per branch

greedy: a split, once made, is never undone

Hyafil and Rivest (1976): building an optimal binary decision tree is NP-complete. Checking one tree is quick, but the number of trees explodes (counted in make.py). So ID3, C4.5 and CART split greedily on the biggest estimated gain: good trees, no optimality guarantee.

4 Surprise: rare events carry more bits



Shannon (1948): "it snowed in Gandhinagar in July" tells you more than "the sun rose". An event carries $I(x) = -\log_2 p(x)$: the minus sign makes it positive (log of $p < 1$ is negative), base 2 gives bits, and a certain event carries 0 bits.

5 Entropy is the expected surprise

outcome	p	$-\log_2 p$	$p \times (-\log_2 p)$
Rainy	0.5	1.00	0.50
Cloudy	0.3	1.74	0.52
Sunny	0.15	2.74	0.41
Snowy	0.05	4.32	0.22

$H = 0.50 + 0.52 + 0.41 + 0.22 = 1.65$ bits

$$H(X) = \mathbb{E}[I(X)] = -\sum_{i=1}^k p(x_i) \log_2 p(x_i)$$

Each Delhi outcome has its own surprise; entropy averages them by probability. H runs from 0 (one certain outcome) to $\log_2 k$ (k equally likely). Take $0 \log_2 0 = 0$: the notebook uses $x \log y$ so a pure node gives 0, not nan.

6 Two classes: 0 bits when pure, 1 bit at 50/50



With $p = p(\text{Yes})$: $H = -p \log_2 p - (1-p) \log_2 (1-p)$. For the full table, $H(S) = -\frac{9}{14} \log_2 \frac{9}{14} - \frac{5}{14} \log_2 \frac{5}{14} = 0.940$. A pure node has low entropy and can be labelled with confidence; a mixed node has high entropy. Good splits make the children purer than the parent.

Information gain picks each split

The whole ID3 run on the tennis table, with every entropy and gain recomputed. Numbers are in bits.

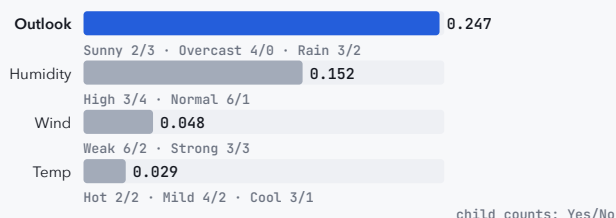
7 Gain is the entropy a split removes



$$\text{Gain}(S, A) = H(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} H(S_v)$$

Each child counts in proportion to its rows. Outlook leaves Sunny and Rain mixed (0.971 each) and Overcast pure.

8 Outlook has the largest gain at the root

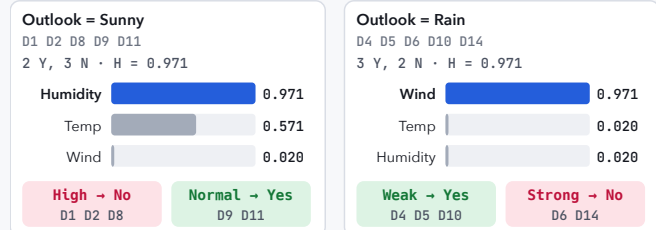


Humidity is second (Normal is 6 Yes, 1 No). Temp barely helps: each Temp branch is close to the parent's mix. The slides print 0.246 and 0.151 after rounding mid-way.

9 ID3 in a few lines

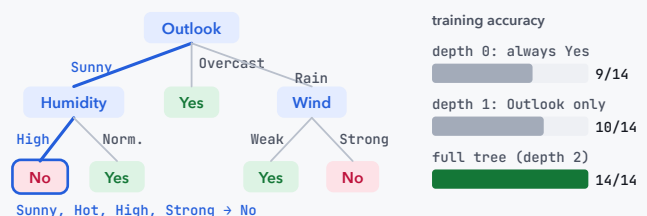
```
ID3(S, attrs):
    if all rows in S share one label:
        return leaf(that label)
    if attrs is empty:
        return leaf(majority label of S)
    A = attr in attrs with max Gain(S, A)
    for each value v of A:
        S_v = rows of S with A = v
        if S_v is empty:
            child v = leaf(majority of S)
        else:
            child v = ID3(S_v, attrs - {A})
```

10 Each branch reruns ID3 on its own rows



Overcast is already a leaf (Yes). Sunny and Rain rerun ID3 without Outlook. In Sunny, Humidity makes both children pure, so its gain equals the branch's entropy, 0.971: the most any split can gain there. In Rain, Wind does the same. The tree now fits all 14 days.

11 Predict by walking the tree from the root



The lecture's query (Sunny, Hot, High humidity, Strong wind) goes Sunny → High → No; Temp and Wind are never read. With a depth limit, a leaf predicts the most common class of its rows: depth 0 always says Yes; depth 1 says No for Sunny, Yes otherwise.

12 The same numbers in pandas

```
def entropy(y): # y: pd.Series
    p = y.value_counts(normalize=True)
    return -(p * np.log2(p)).sum()

def gain(df, a, y="Play"):
    w = df[a].value_counts(normalize=True)
    h = df.groupby(a)[y].apply(entropy)
    return entropy(df[y]) - (w * h).sum()
```

entropy(df["Play"]) gives 0.9403, as in the notebook. gain gives 0.247, 0.152, 0.048, 0.029.

13 Pause and think

Entropy of 14 Yes, 0 No? 0 bits: there is nothing to decide.

Day (D1 to D14) as an attribute? Gain 0.940, the maximum, since each branch holds one row; the tree just memorises.

Three equally likely classes? $\log_2 3 = 1.585$ bits, the most for 3 outcomes.

Does ID3 find the most accurate tree? No guarantee; it never revisits a split.

Why Outlook first? Its Overcast branch is pure, so it adds no entropy.

Wind's gain at the root vs inside Rain? 0.048 vs 0.971: a split's value depends on the rows it sees.

entropy = expected surprise · gain = parent entropy – weighted child entropy · ID3 splits greedily, then recurses

Next, on the decision trees II sheet: real-valued inputs and outputs, Gini, depth and pruning. Source: lecture slides "Decision trees" (slides), notebooks entropy and decision-tree-discrete-input-discrete-output; every number recomputed in NumPy.